

Button Control

Button Control

1. Learning Objectives
2. Hardware Setup
 - Independent Button Principle
 - Button Debouncing
3. Experiment Steps
 1. Open SYSCONFIG Configuration Tool
 2. Configure Pin Mode
 3. Write Program
 4. Compile
4. Program Analysis
5. Experimental Phenomena

1. Learning Objectives

1. Learn the basic working principle of buttons on the MSPM0G3519 motherboard.
2. Use the button module to control the onboard LED light on/off.

2. Hardware Setup

Independent Button Principle

Independent buttons are typically implemented through **mechanical switches**. Their hardware structure consists of a button, connecting wires, and pull-up/pull-down resistors. When the button is closed, the circuit is connected and the signal level changes; when the button is released, the circuit is disconnected and the signal returns to its initial state.

Button Debouncing

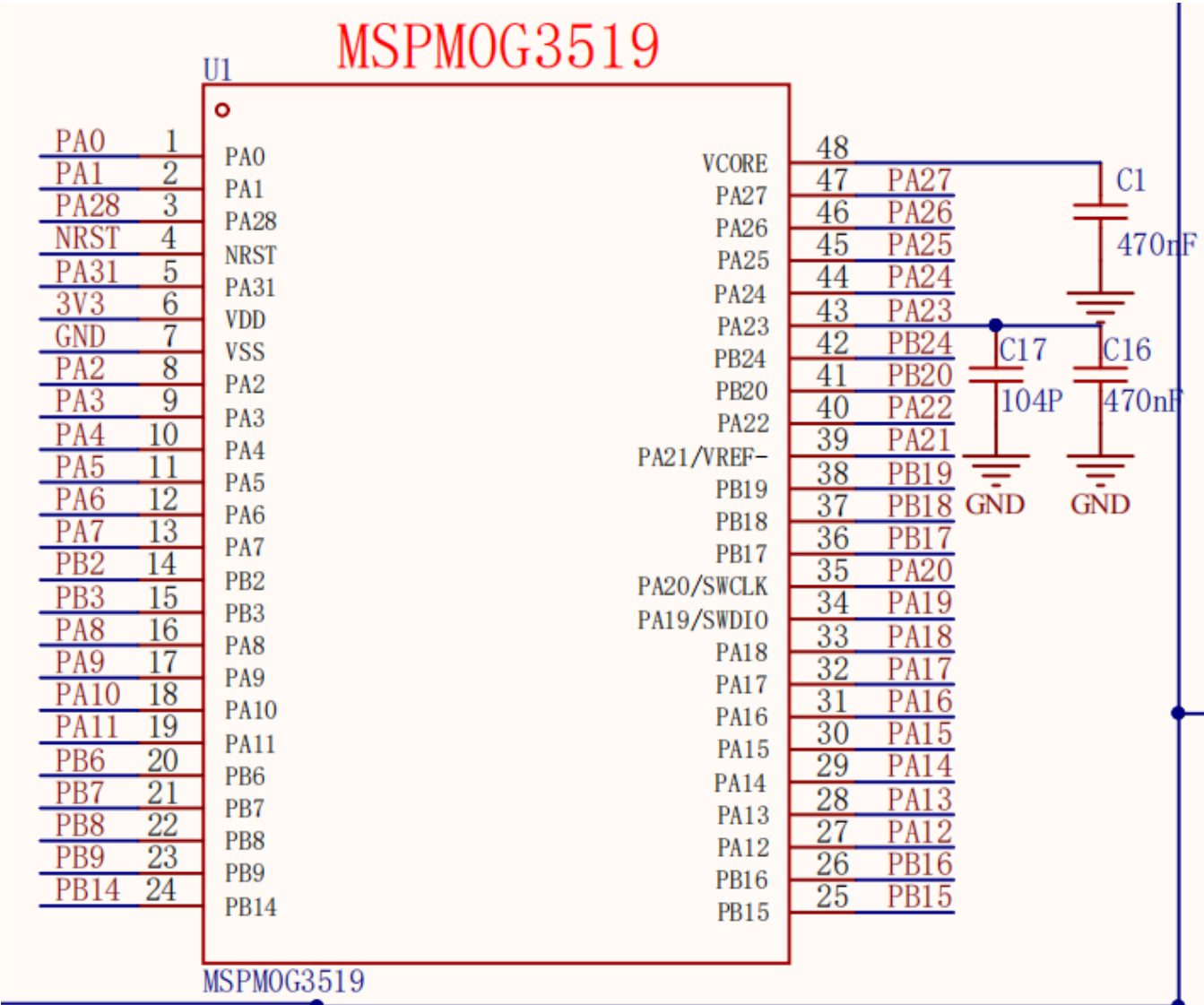
The mechanical elasticity of buttons causes the signal to jump multiple times in a short period when closing or releasing. This is called **button bouncing**. It is usually eliminated through **software delay debouncing** or hardware **RC filter circuit**.

This course does not require additional hardware. You can directly use the onboard function button on the MSPM0G3519 development board.

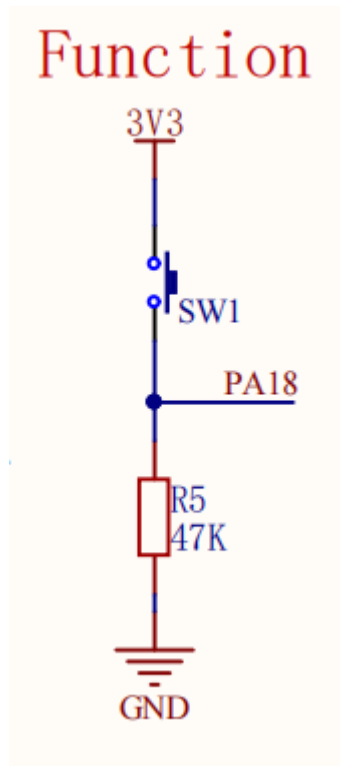
In the development board schematic, the PA18 pin is connected to a button. One end of the button is connected to high level (3.3V), and the other end is connected to the GPIO pin, while also being connected to ground through a 47kΩ pull-down resistor R5. When the button is not pressed, the PA18 pin level is pulled low to 0V due to the pull-down resistor. When the button is pressed, the PA18 pin is directly connected to high

level 3.3V, thereby achieving high and low level changes. The development board determines whether the button is pressed by detecting the level status of the PA18 pin. This design is simple and stable, effectively avoiding signal interference caused by floating pins.

MSPM0G3519 Main Control Diagram



Button Part Schematic

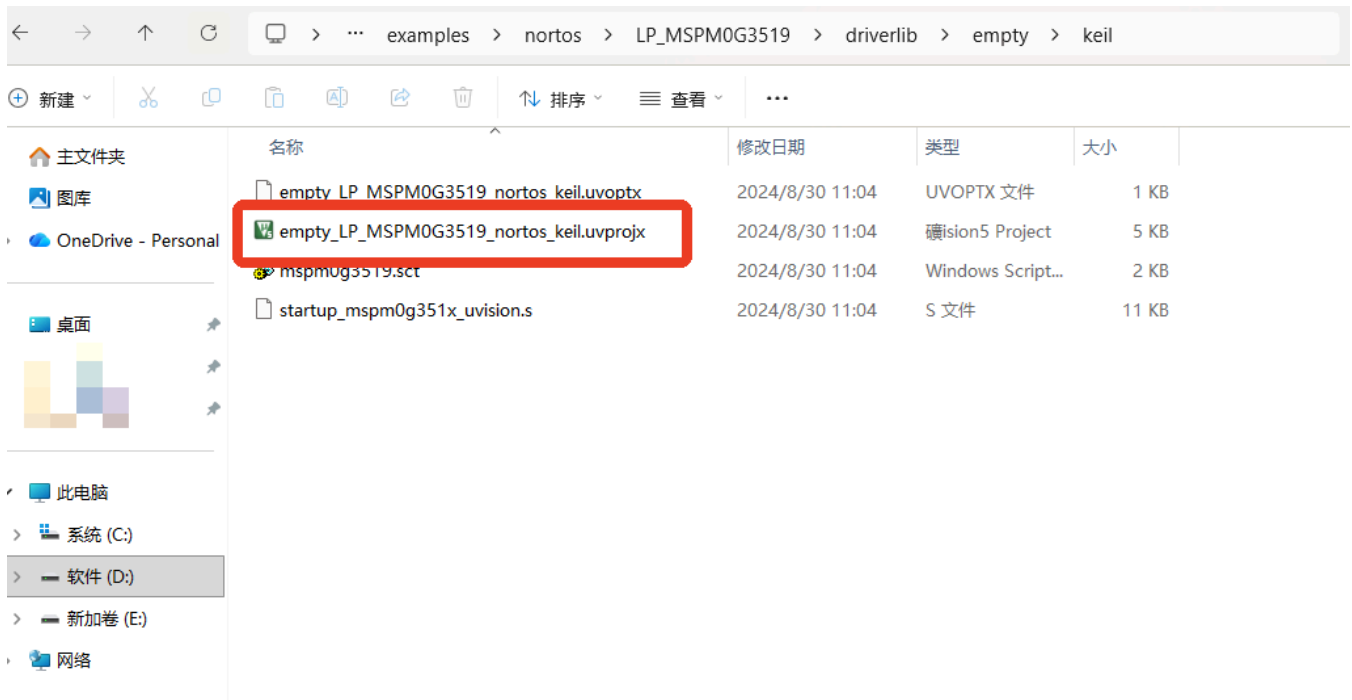


3. Experiment Steps

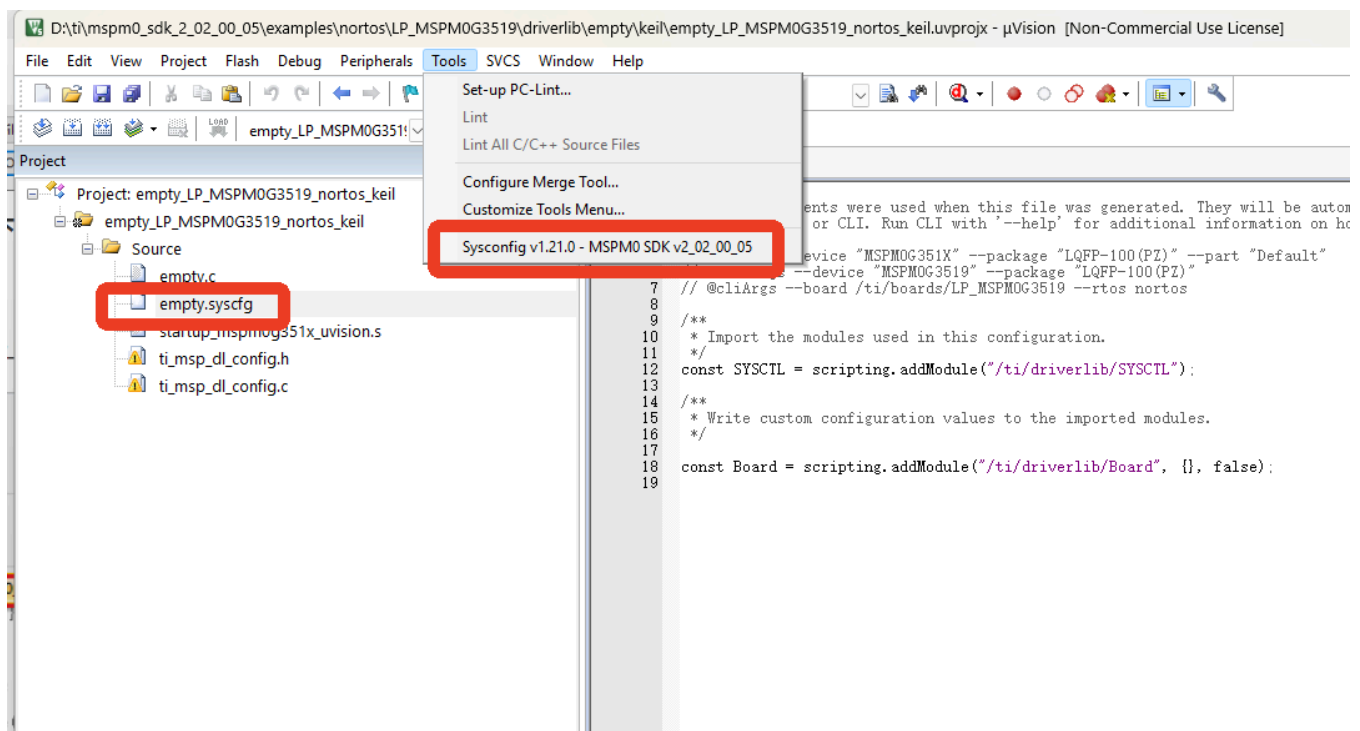
In microcontrollers, pin input mode means configuring a certain pin as an input function to receive changes in external signals. The working principle of buttons is based on this: by setting the pin connected to the button as input mode, the microcontroller can detect level changes on the pin. When the button is not pressed, the pin level remains low; when the button is pressed, the pin level becomes high. This high and low level change is the embodiment of the button input signal, and it is also the basis for the microcontroller to determine button status. Therefore, the process of receiving external signals is actually the process of detecting button high and low level changes through input mode.

1. Open SYSCONFIG Configuration Tool

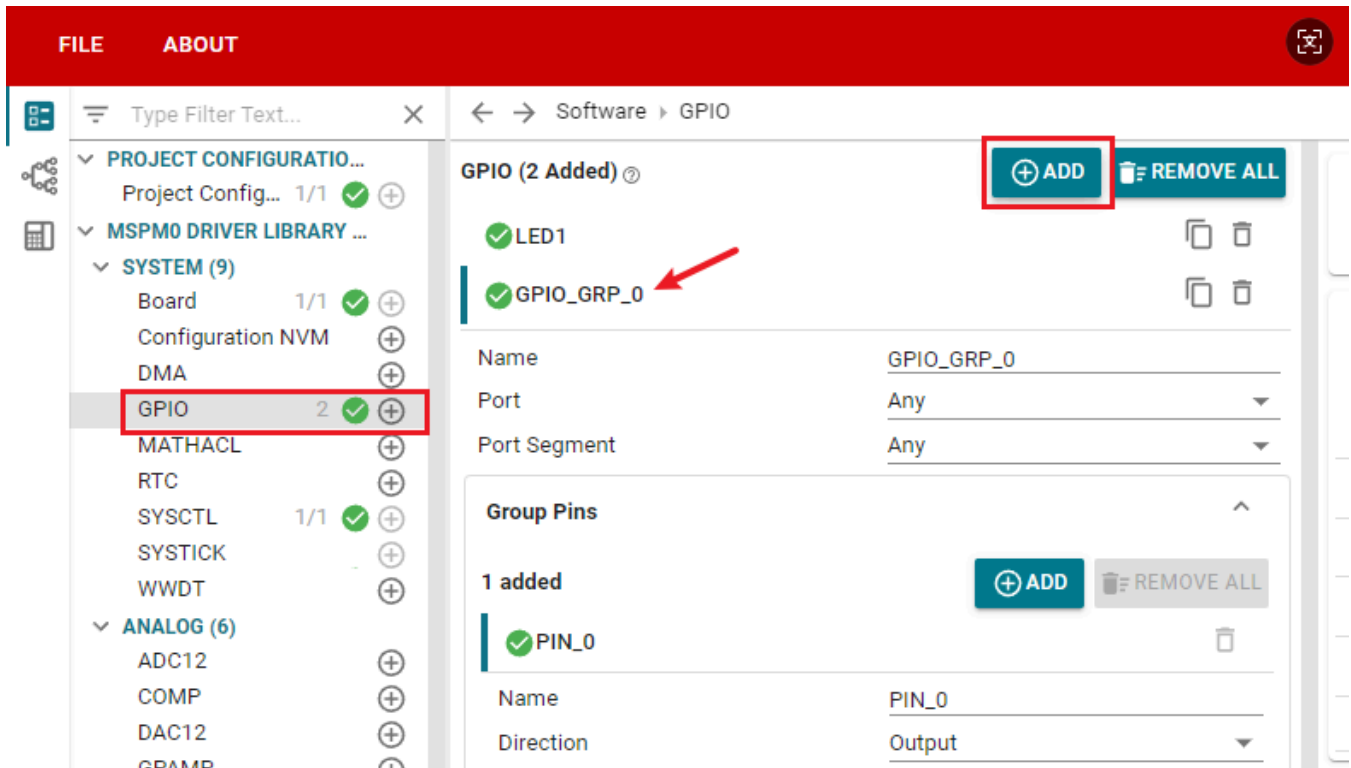
Open the empty project from the SDK in KEIL.



After selecting and opening it, open the empty.syscfg file in the KEIL interface. **With the empty.syscfg file open**, then open the SYSCONFIG GUI interface from the Tools menu.



In SYSCONFIG, select MCU peripherals on the left, find the GPIO option and click to enter. Click ADD in GPIO to add a GPIO group. This GPIO_GRP_0 is the newly added one.



2. Configure Pin Mode

Configure pin parameters, name the GPIO instance `KEY`. Since it is connected to PA18, Port selects `PORTA`.

Set the pin name to `PIN_18`, set to input mode `Input`. Internal Resistor sets to pull-down resistor `Pu11-Down Resistor`. Assigned Pin sets the controlled pin to 18.

✓ KEY

Name

KEY

Port

PORTA

Port Segment

Any

Group Pins

1 added

+

 ADD

REMOVE ALL

✓ PIN_18

Name

PIN_18

Direction

Input

IO Structure [?]

Any

Digital IOMUX Features

Internal Resistor

Pull-Down Resistor

Invert

Disabled

Input Filter

Disabled

Enable Fast-Wake

☐

Override Pin Hysteresis Default

☐

Hysteresis Control

Disabled

Wakeup Logic

Disabled

Assigned Port

PORTA

Assigned Port Segment

Any

Assigned Pin

18

Interrupts/Events

LaunchPad-Specific Pin

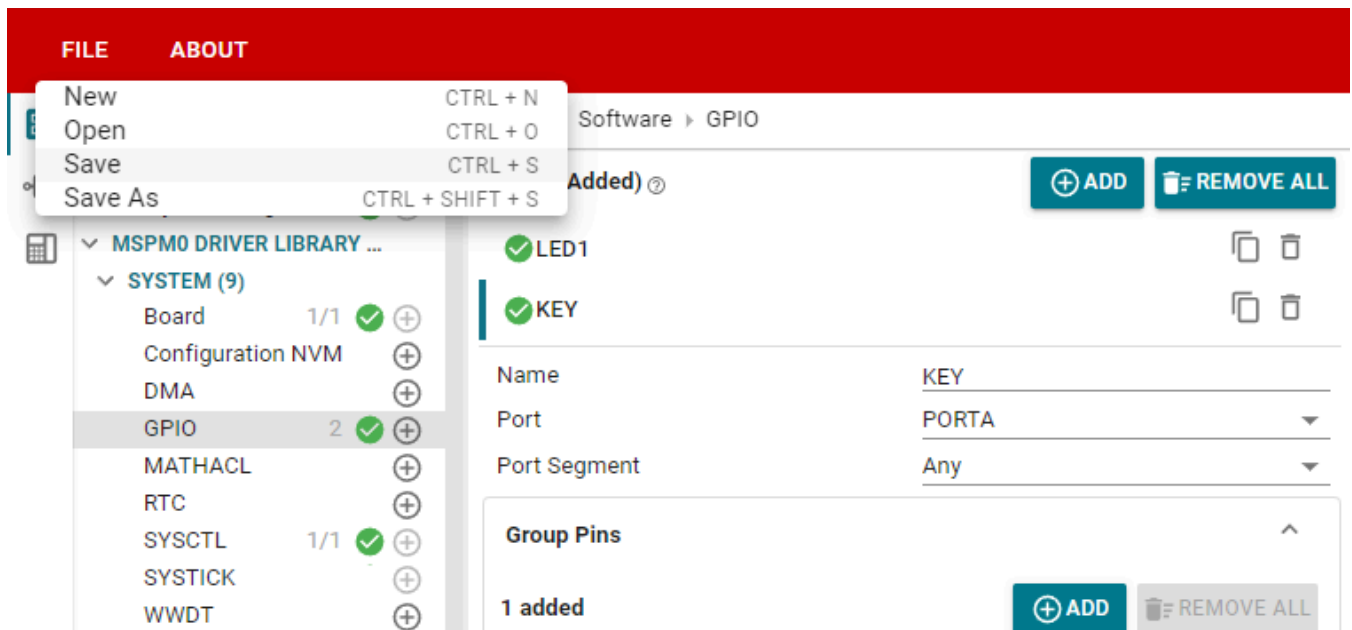
No Shortcut Used

PinMux Peripheral and Pin Configuration

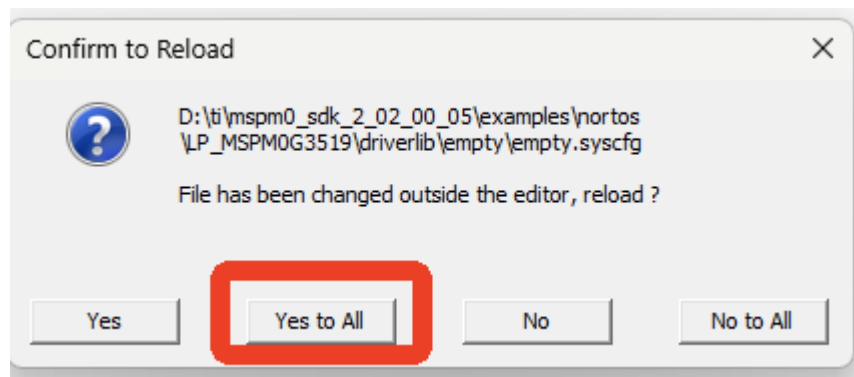
PIN_18

Any(PA18/11)

Click SAVE to save the SYSCONFIG configuration, then close SYSCONFIG and return to KEIL.



Select **Yes to All** in the popup.



We also need to confirm whether `ti_msp_dl_config.c` and `ti_msp_dl_config.h` files are updated. Simply compile, and the compilation will automatically update to KEIL. If not updated, we can also copy the file content from `SYSCONFIG`.

3. Write Program

In the `empty.c` file, write the following code

```
#include "ti_msp_dl_config.h"

int main(void)
{
    SYSCFG_DL_init();

    while (1)
    {
        // If the pin value read is greater than 0, it means that the PA18 pin is
        high level.
        if( DL_GPIO_readPins(KEY_PORT, KEY_PIN_18_PIN) > 0 )
        {
            DL_GPIO_setPins(LED1_PORT, LED1_PIN_2_PIN); //LED control output
            high level
        }
    }
}
```

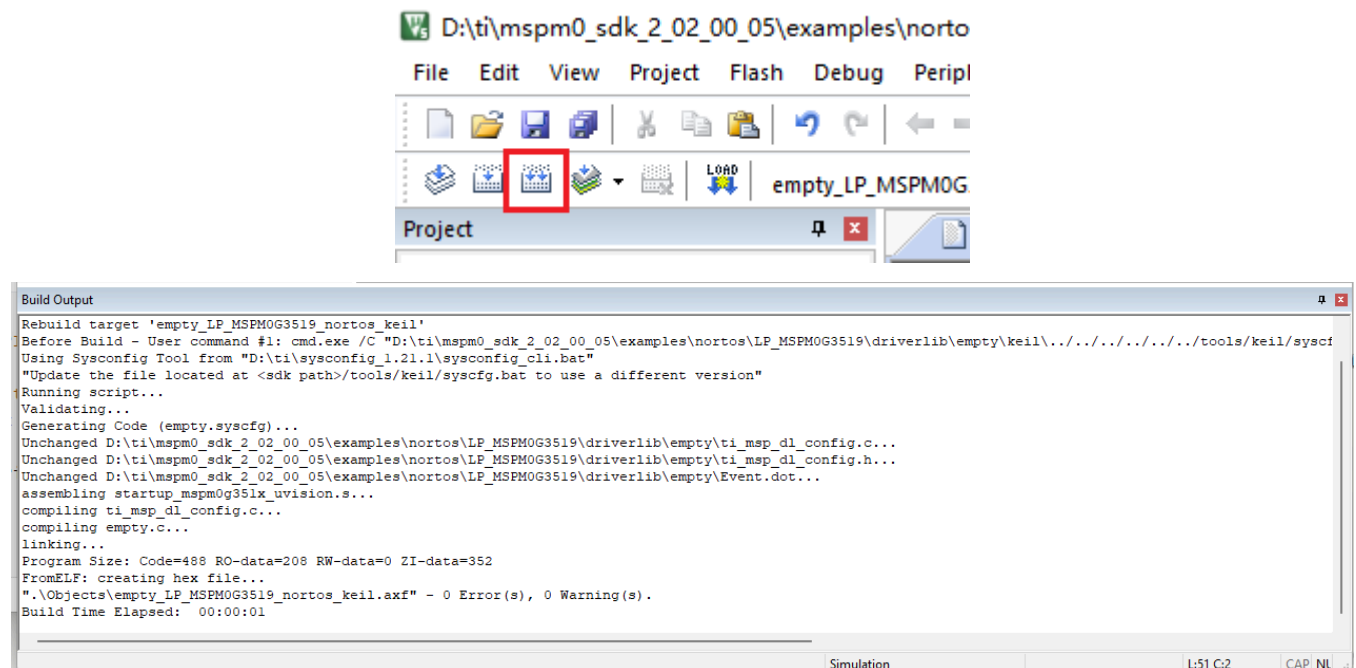
```

    }
    else// If the PA18 pin is low
    {
        DL_GPIO_clearPins(LED1_PORT,LED1_PIN_2_PIN);//LED control output low
        level
    }
}
}

```

4. Compile

Click the Rebuild icon. The following prompt indicates compilation is complete with no errors.



4. Program Analysis

- dl_gpio.h

```

/**
 * @brief      Read a group of GPIO pins
 *
 * @param[in]  gpio  Pointer to the register overlay for the peripheral
 * @param[in]  pins  Pins to read. Bitwise OR of @ref DL_GPIO_PIN.
 *
 * @return     The pins (from the selection) that are currently high
 *
 * @retval     Bitwise OR of @ref DL_GPIO_PIN of pins that are currently high
 *             from the input selection.
 */
__STATIC_INLINE uint32_t DL_GPIO_readPins(GPIO_Regs* gpio, uint32_t pins)
{
    return (gpio->DIN31_0 & pins);
}

```

uint32_t DL_GPIO_readPins(GPIO_Regs* gpio, uint32_t pins) is used to read the level status of specified GPIO pins. By reading the value of the GPIO register, it obtains the status of the specified pin. Input the GPIO port and pin number to read, and return the level status of that pin.

gpio: The GPIO port to read.

pins: The pin number to read.

- empty.c

```
int main(void)
{
    SYSCFG_DL_init();

    while (1)
    {
        // If the pin value read is greater than 0, it means that the PA18 pin is
        high level.
        if( DL_GPIO_readPins(KEY_PORT, KEY_PIN_18_PIN) > 0 )
        {
            DL_GPIO_setPins(LED1_PORT, LED1_PIN_2_PIN); //LED control output
            high level
        }
        else// If the PA18 pin is low
        {
            DL_GPIO_clearPins(LED1_PORT, LED1_PIN_2_PIN); //LED control output low
            level
        }
    }
}
```

This code controls the LED by continuously detecting the level status of pin PA18 (button input pin) to control the output level of LED1_PIN_2 (LED control pin). When PA18 is high, the LED turns on (LED pin outputs high level); when PA18 is low, the LED turns off (LED pin outputs low level).

5. Experimental Phenomena

After the program is downloaded, you can control the LED light on/off with the button. Press the button, the LED turns on; release the button, the LED turns off.

